SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

그래프의 최소 비용 문제

Python

학습시작

아래 문장은 컴퓨터 입장에서 본 그래프의 최소 비용 문제입니다.

1. 복잡하게 얽힌 수 많은 정점과 간선을 보고, 전체를 연결하는 가장 [ ] 방법을 찾습니다. 단, 어디에도 [ ] 이 생기면 안됩니다.
2. 때로는 정점에서 출발하고, 어떤 때는 간선을 하나하나 [ ] 하면서 최종적으로 [ ] 이 가장 적은 연결망을 찾습니다.

- 복잡한 네트워크나 지도, 연결 관계를 가장 효율적으로 구성하는 방법을 익힐 수 있습니다.
- 최단 비용, 최단 경로, 효율적인 데이터 흐름 설계에 필요한 핵심 알고리즘 사고력을 키울 수 있습니다.

학습목표

- 최소 신장 트리의 의미를 이해한다.
- 탐욕 기법을 이용해서 그래프에서 최소 신장 트리를 찾는 프림과 크루스칼 알고리즘의 동작원리를 이해한다.
- 단일 시작점 최단 경로 문제를 해결하는 다익스트라 알고리즘에 대해 학습한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

최소 비용 신장 트리

- 최소 비용 신장 트리 (MST)
- Prim 알고리즘
- Kruskal 알고리즘

최단 경로

- Dijkstra 알고리즘

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

최소 비용 신장 트리

최소 비용 신장 트리 (MST)

대표적인 트리 문제

- 그래프에서 최소 비용 문제

 - 모든 정점을 연결하는 간선들의 가중치의 합이 최소가 되는 트리를 찾습니다.

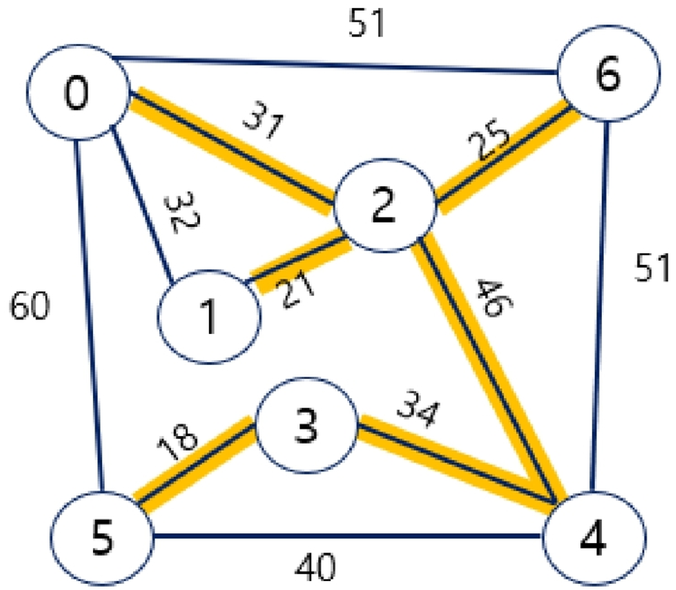
 - 두 정점 사이의 비용이 최소인 경로를 찾습니다.

- 신장 트리

 - N개의 정점으로 이루어진 무방향 그래프에서 N개의 정점과 N-1개의 간선으로 이루어진 트리입니다.

- 최소 신장 트리 (MST, Minimum Spanning Tree)

 - 무방향 가중치 그래프에서 신장 트리를 구성하는 간선들의 가중치 합이 최소인 신장 트리입니다.



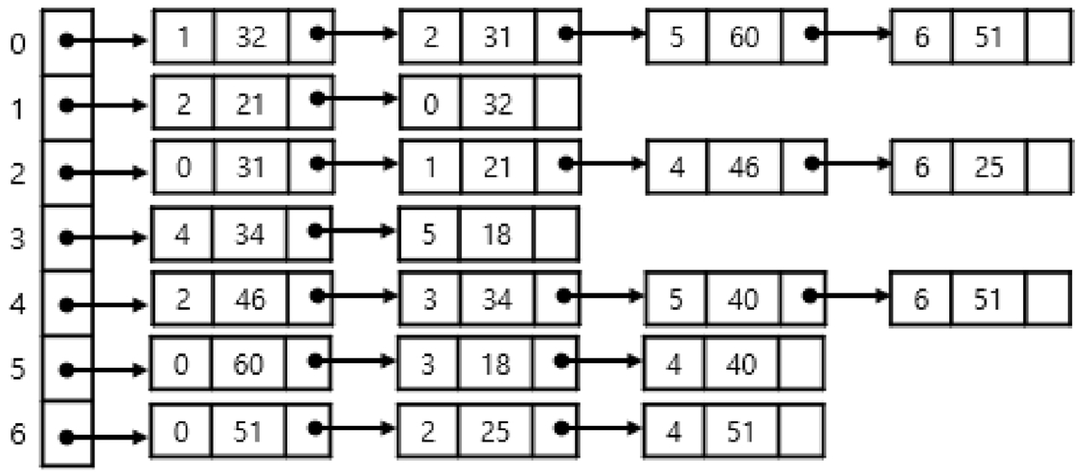
<그림1_SW문제해결 응용_그래프의 최소비용 문제_그래프에서의 MST>

그래프의 표현

- 인접행렬
- 인접리스트

	0	1	2	3	4	5	6
0	-	32	31	-	-	60	51
1	32	-	21	-	-	-	-
2	31	21	-	-	46	-	25
3	-	-	-	-	34	18	-
4	-	-	46	34	-	40	51
5	60	-	-	18	40	-	-
6	51	-	25	-	51	-	-

<그림2_SW문제해결 응용_그래프의 최소비용 문제_가중치 인접행렬>



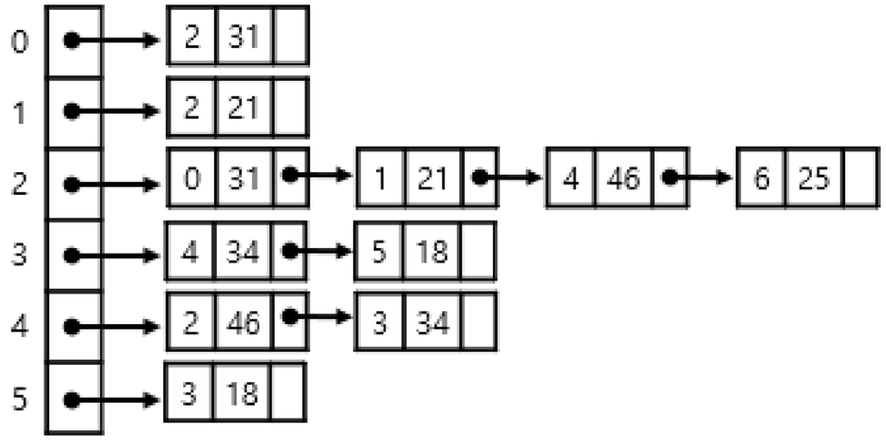
<그림3_SW문제해결 응용_그래프의 최소비용 문제_가중치 인접리스트>

MST 표현

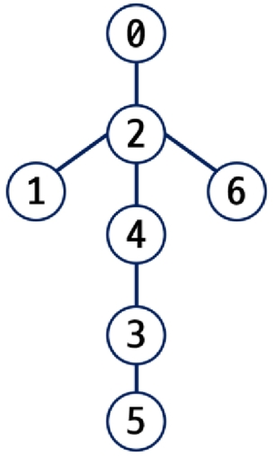
- 그래프 (그림1)
- 간선들의 배열 (그림4)
- 인접 리스트 (그림5)
- 트리 (그림6)
- 부모 자식 관계와 가중치에 대한 배열 (그림6)

	0	1	2
0	0	2	31
1	2	1	21
2	2	6	25
3	2	4	46
4	4	3	34
5	3	5	18

<그림4_SW문제해결 응용_그래프의 최소비용 문제_MST 표현4>



<그림5_SW문제해결 응용_그래프의 최소비용 문제_MST 표현5>



	0	1	2	3	4	5	6
st	0	2	0	4	2	3	2
Val	0	21	31	34	46	18	25

<그림6_SW문제해결 응용_그래프의 최소비용 문제_MST 표현6>

MST 찾기

- Prim 알고리즘
- Kruskal 알고리즘

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Prim 알고리즘

Prim 알고리즘이란?

- 하나의 정점에 연결된 간선들 중에서, 하나씩 선택하면서 MST를 만들어 가는 방식

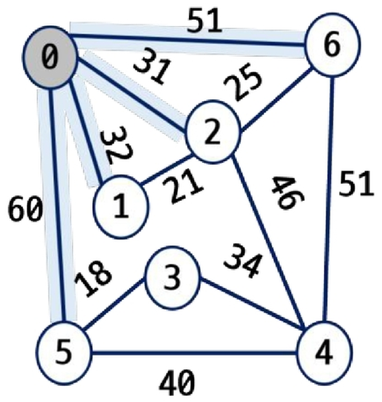
1. 임의 정점을 하나 선택해서 시작
2. 선택한 정점과 인접하는 정점들 중의 최소 비용의 간선이 존재하는 정점을 선택
3. 모든 정점이 선택될 때까지 앞의 과정을 반복

- 서로소인 2개의 집합 정보를 유지

- 트리 정점들(tree vertices) - MST를 만들기 위해 선택된 정점들 입니다.
- 비트리 정점들(nontree vertices) - 선택되지 않은 정점들 입니다.

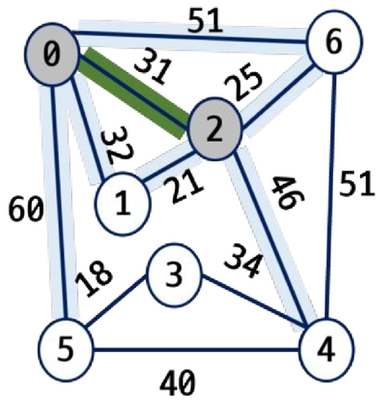
Prim 알고리즘 적용 예

1



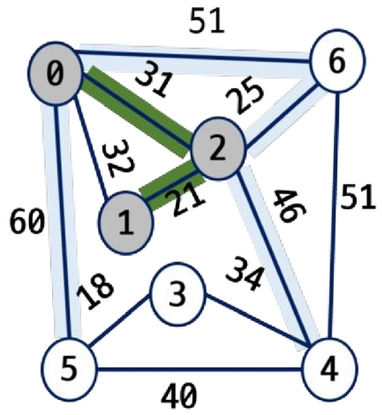
간선들> 0-1 0-2 0-5 0-6

2



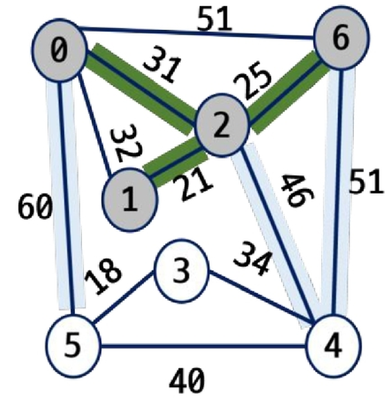
간선들> 0-1 0-5 0-6 1-2 2-4 2-6

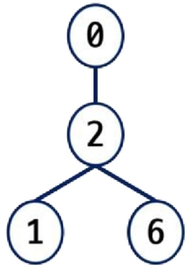
3



간선들> 0-5 0-6 2-4 2-6

4



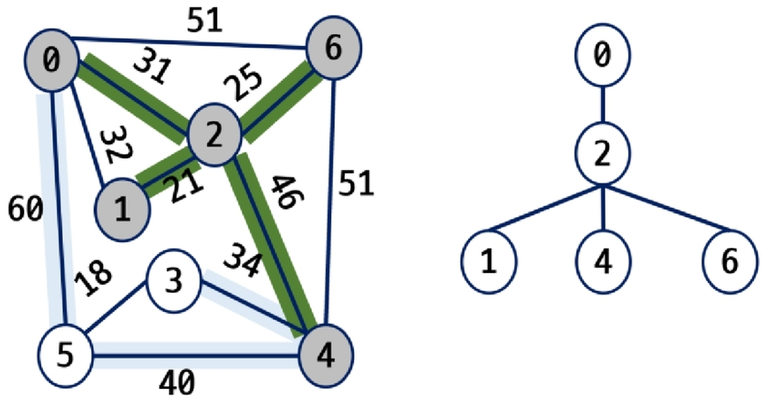


간선들> 0-5 2-4 6-4

<그림7_SW문제해결 응용_그래프의 최소비용 문제_Prim 알고리즘1>

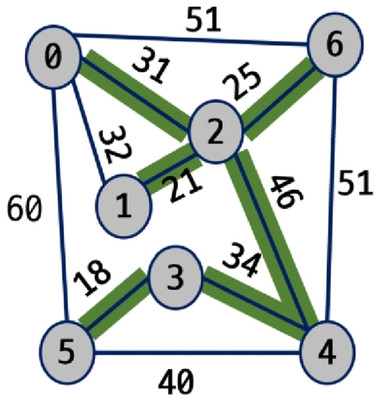
Prim 알고리즘 적용 예

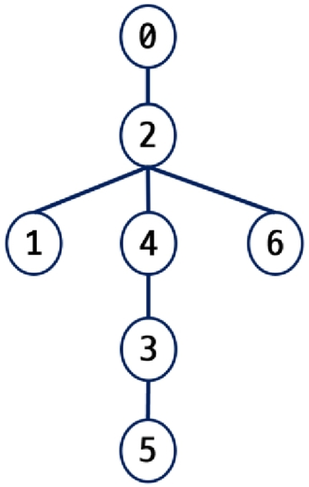
5



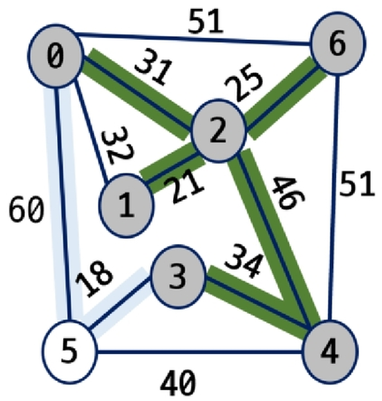
간선들> 0-5 4-3 4-5

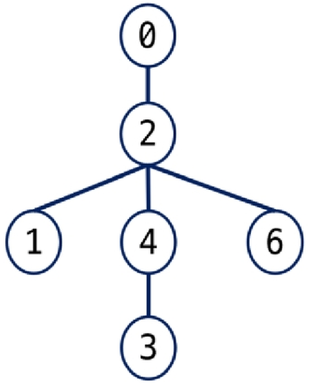
7





6





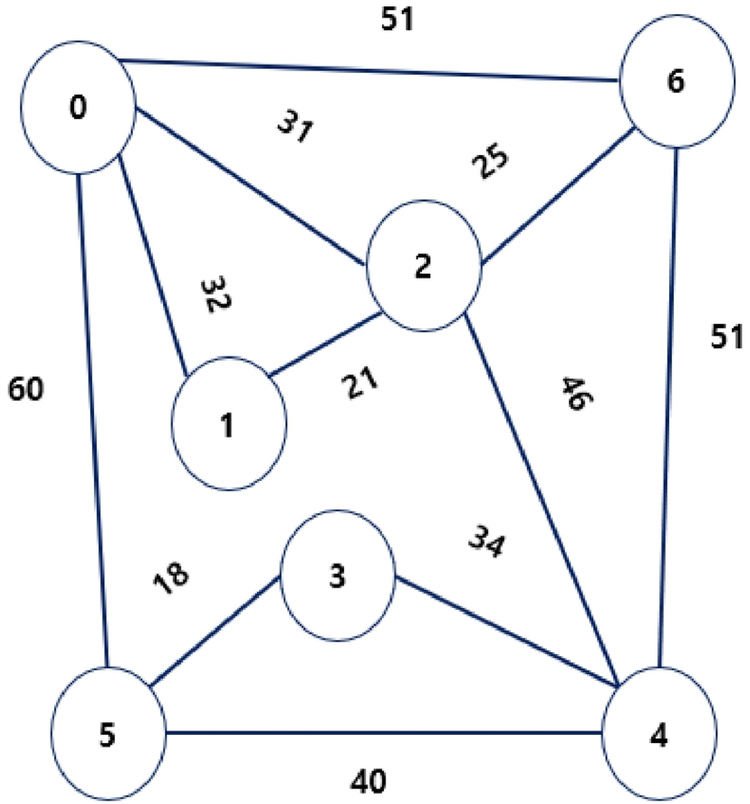
간선들> 0-5 3-5 4-5

	0	1	2	3	4	5	6
st	0	2	0	4	2	3	2
val	0	21	31	34	46	18	25

<그림8_SW문제해결 응용_그래프의 최소비용 문제_Prim 알고리즘2>

Prim 알고리즘 연습

- 코드 : 1.prim.py



<그림9_SW문제해결 응용_그래프의 최소비용 문제_Prim 알고리즘3>

Prim 알고리즘 2

- MST에 아직 포함되지 않은 정점을 중심으로 계산하는 방식

```
MST_PRIM(G, r)                              // G: 그래프, r:시작 정점
    FOR u in G.V
        u.key ← ∞                           // u.key: u에 연결된 간선중 최소 가중치
        u.π ← NULL                          // u.π : 트리에서 u의 부모
    r.key ← 0
    Q ← G.V                                 // 우선순위 Q에 모든 정점 넣는다.
    WHILE Q ≠ 0                             // 빈 Q 가 아닐동안 반복
        u ← Extract_MIN(Q)                  // key 값이 가장 작은 정점 가져오기
        FOR v in G.Adj[u]                   // u의 인접 정점들
            IF v ∈ Q AND w(u, v) < v.key     // Q에 있는 v의 key값 갱신
                v.π ← u
                v.key ← w(u, v)
```

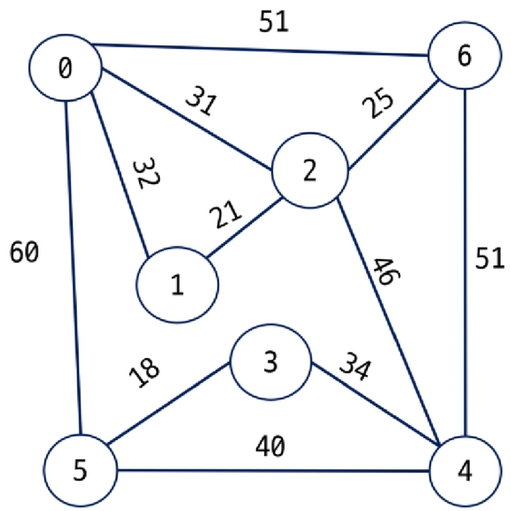
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Kruskal 알고리즘

간선을 하나씩 선택해서 MST를 찾는 알고리즘

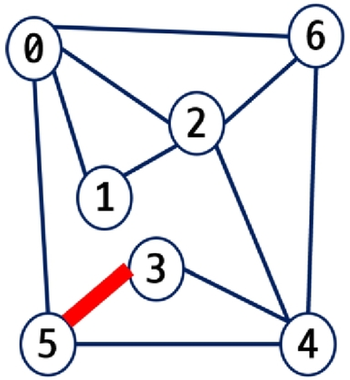
1. 최초, 모든 간선을 가중치에 따라 오름차순으로 정렬
2. 가중치가 가장 낮은 간선부터 선택하면서 트리를 증가시킴
   - 사이클이 존재하면 다음으로 가중치가 낮은 간선 선택
3. n-1개의 간선이 선택될 때까지 2를 반복



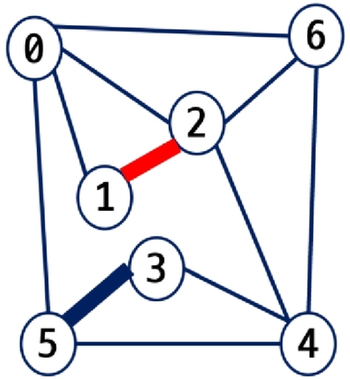
간선	가중치
5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60

<그림10_SW문제해결 응용_그래프의 최소비용 문제_Kruskal 알고리즘1>

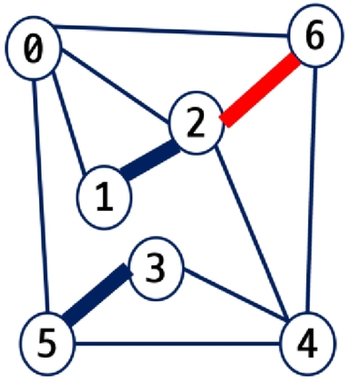
Kruskal 알고리즘 적용 예시



5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60



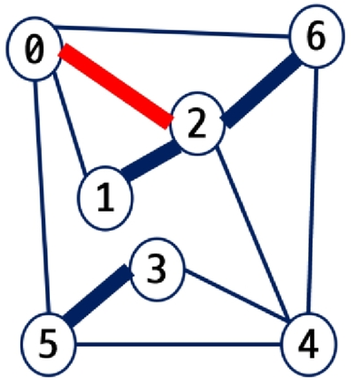
5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60



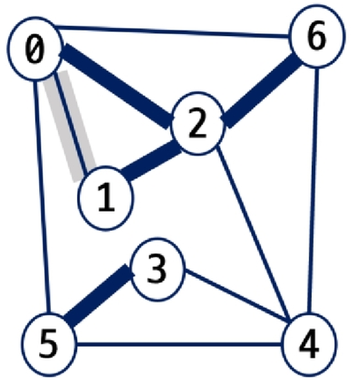
5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60

<그림11_SW문제해결 응용_그래프의 최소비용 문제_Kruskal 알고리즘2>

Kruskal 알고리즘 적용 예시

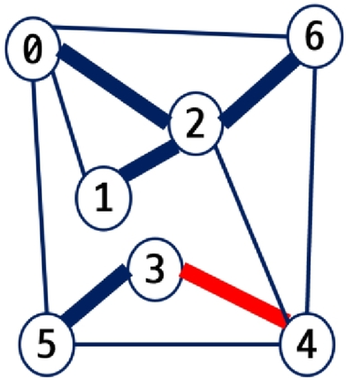


5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60



싸이클 존재
다음 간선 선택

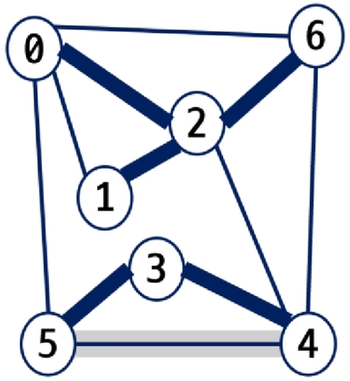
5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60



5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60

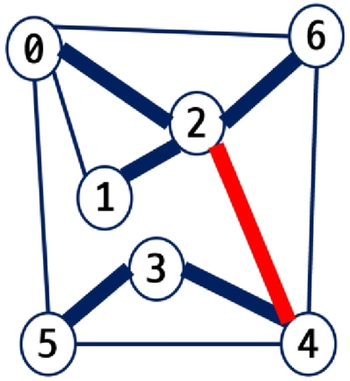
<그림12_SW문제해결 응용_그래프의 최소비용 문제_Kruskal 알고리즘3>

Kruskal 알고리즘 적용 예시



싸이클 존재
다음 간선 선택

5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60



5-3	18
1-2	21
2-6	25
0-2	31
0-1	32
3-4	34
5-4	40
2-4	46
0-6	51
4-6	51
0-5	60

그래프의 정점의 수 N = 7

N-1 = 6 개의 간선이 선택됨

<그림13_SW문제해결 응용_그래프의 최소비용 문제_Kruskal 알고리즘4>

Kruskal 알고리즘

```
MST-KRUSKAL(G, w)
    A ← 0                          // 0: 공집합
    FOR vertex v in G.V            // G.V: 그래프의 정점 집합
        Make_Set(v)                // G.E: 그래프의 간선 집합

    G.E에 포함된 간선들을 가중치 w 에 의해 정렬

    FOR 가중치가 가장 낮은 간선 (u, v) ∈ G.E 선택(n-1개)
        IF  Find_Set(u) ≠ Find_Set(v)
            A  ←  A ∪ {(u, v)}
            Union(u, v);

    RETURN A
```

최단 경로

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Dijkstra 알고리즘

최단 경로란?

최단 경로

간선의 가중치가 있는 그래프에서 두 정점 사이의
경로들 중에 간선의 가중치의 합이 최소인 경로

최단 경로 알고리즘의 종류

1. 하나의 시작 정점에서 끝 정점까지의 최단 경로
   - 다익스트라(Dijkstra) 알고리즘 - 음의 가중치를 허용하지 않음
   - 벨만-포드(Bellman-Ford) 알고리즘 - 음의 가중치 허용

2. 모든 정점들에 대한 최단 경로
   - 플로이드-워샬(Floyd-Warshall) 알고리즘

Dijkstra 알고리즘이란?

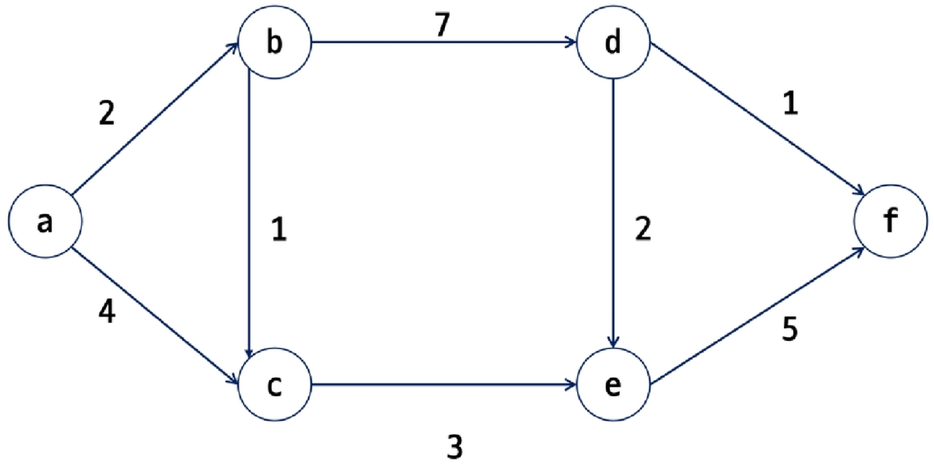
Dijkstra 알고리즘

Dijkstra algorithm

시작 정점에서 거리가 최소인 정점을 선택해 나가면서
최단 경로를 구하는 알고리즘

Dijkstra 알고리즘

- 시작 정점(s)에서 끝 정점(t)까지의 최단 경로에 정점 x가 존재함
  - 예) a에서 f까지 가는 최단 경로(비용 10)에 d가 존재합니다.
- 이때, 최단 경로는 s에서 x까지의 최단 경로와 x에서 t까지의 최단 경로로 구성됨
  - 예) a에서 d까지 가는 최단 경로(비용 9)와 d에서 f까지 가는 최단 경로(비용1)로 구성됩니다.
- 탐욕 기법을 사용한 알고리즘으로 MST의 Prim 알고리즘과 유사함



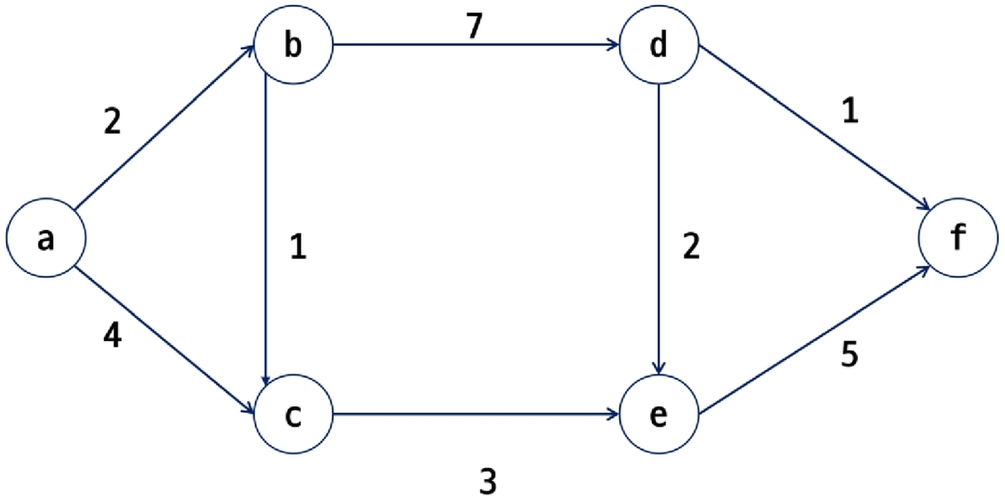
<그림14_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘1>

Dijkstra 알고리즘

```
s: 시작 정점, A: 인접 행렬, D: 거리
V: 정점 집합, U: 선택된 정점 집합

Dijkstra(s, A, D)
    U = { s };

    FOR 모든 정점 v
        D[v]  ← A[s][v]

    WHILE U ≠ V
        D[w]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w}

        FOR w 에 인접한 모든 정점 v
                D[v] ← min(D[v], D[w] + A[w][v])
```

Dijkstra 알고리즘 적용 예시

- 정점 a에서 각 정점으로 가는 최소비용은?



<그림14_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘1>

인접 행렬

	a	b	c	d	e	f
a	0	2	4	∞	∞	∞
b	∞	0	1	7	∞	∞
c	∞	∞	0	∞	3	∞
d	∞	∞	∞	0	2	1
e	∞	∞	∞	∞	0	5
f	∞	∞	∞	∞	∞	0

D

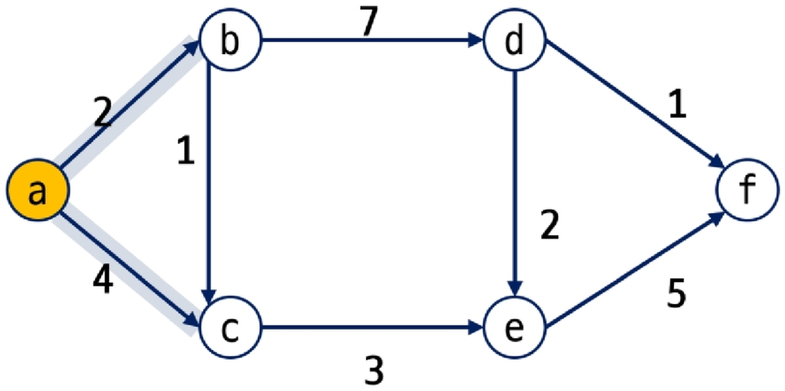
a	b	c	d	e	f
∞	∞	∞	∞	∞	∞

U

<그림15_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘2>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)
    U = {s };

    FOR 모든 정점 v
        D[v ] ← A[s ][v ]

    WHILE U ≠ V
        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림16_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘3>

인접 행렬 A

	a	b	c	d	e	f
a	0	2	4	∞	∞	∞

D

a	b	c	d	e	f
0	2	4	∞	∞	∞

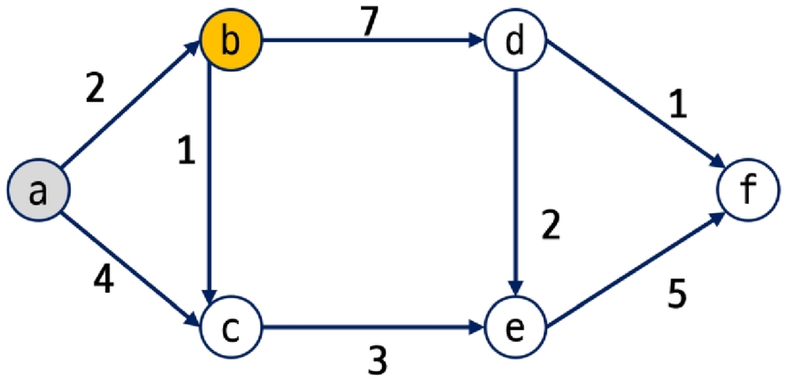
U

a

<그림17_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘4>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)
    U = {s };

    FOR 모든 정점 v
        D[v ] ← A[s ][v ]

    WHILE U ≠ V
        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림18_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘5>

D

a	b	c	d	e	f
0	2	4	∞	∞	∞

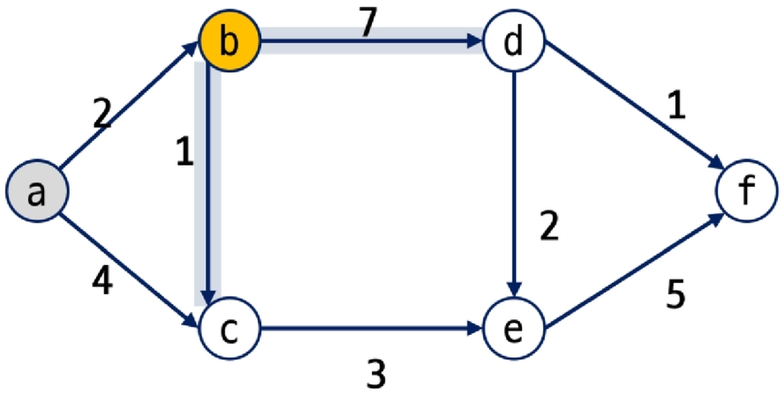
U

a	b

<그림19_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘6>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)
    U = {s };

    FOR 모든 정점 v
        D[v ] ← A[s ][v ]

    WHILE U ≠ V
        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림20_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘7>

D

a	b	c	d	e	f
0	2	4	∞	∞	∞

a	b	c	d	e	f
0	2	3	9	∞	∞

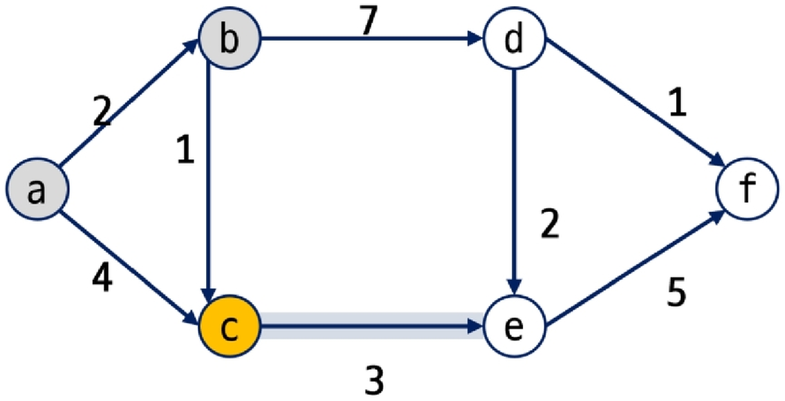
U

a	b

<그림21_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘8>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)

    …

        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림22_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘9>

D

a	b	c	d	e	f
0	2	3	9	∞	∞

a	b	c	d	e	f
0	2	3	9	6	∞

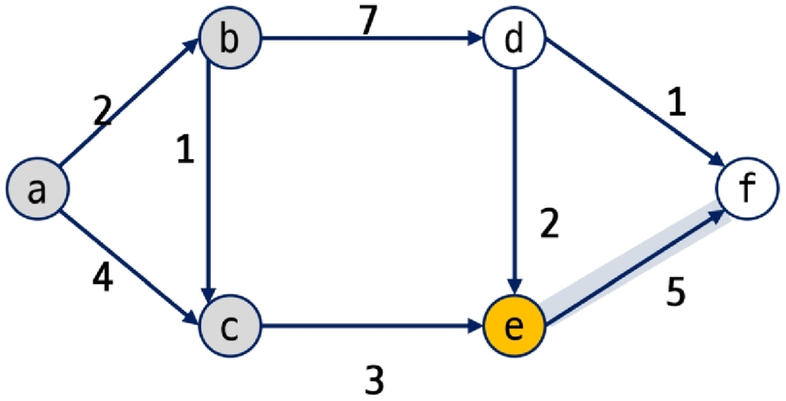
U

a	b	c

<그림23_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘10>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)

    …

        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림24_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘11>

D					
a	b	c	d	e	f
0	2	3	9	6	∞

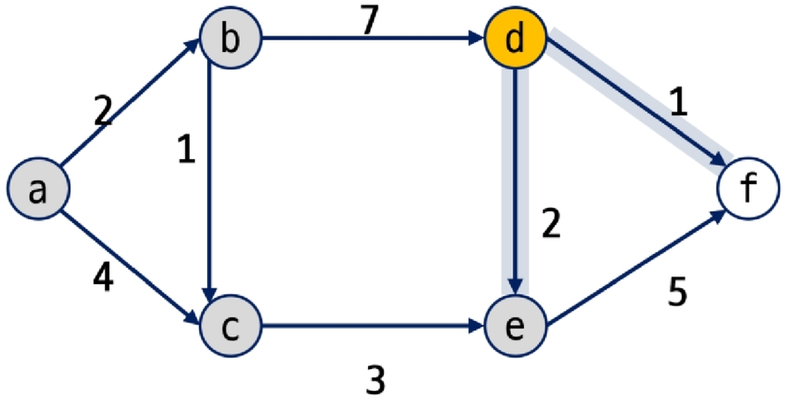
a	b	c	d	e	f
0	2	3	9	6	11

U			
a	b	c	e

<그림25_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘12>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)

    …

        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림26_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘13>

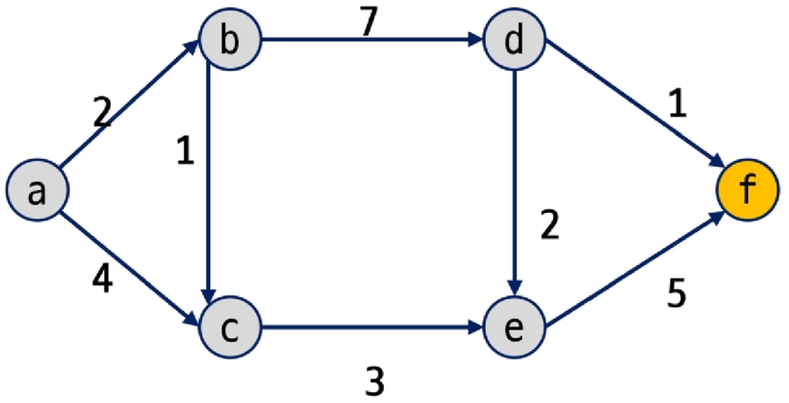
D					
a	b	c	d	e	f
0	2	3	9	6	11
a	b	c	d	e	f
0	2	3	9	6	10

U				
a	b	c	e	d

<그림27_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘14>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)

    …

        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림28_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘15>

D

a	b	c	d	e	f
0	2	3	9	6	10

a	b	c	d	e	f
0	2	3	9	6	10

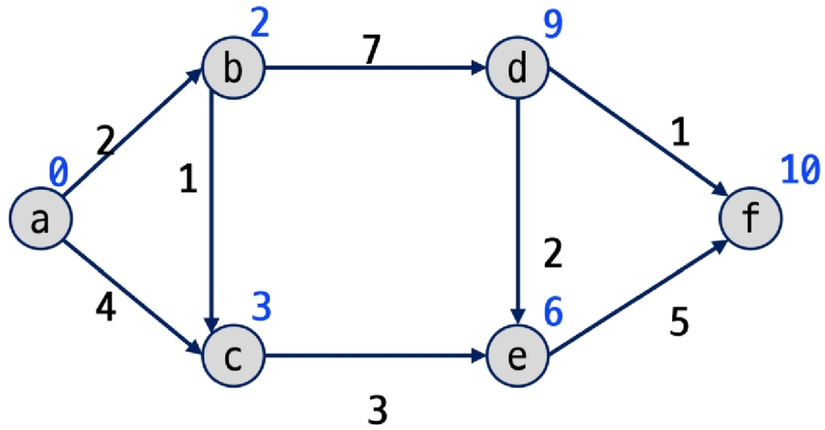
U

a	b	c	d	e	f

<그림29_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘16>

Dijkstra 알고리즘 적용 예시

```
Dijkstra(s, A, D)
    U = {s };

    FOR 모든 정점 v
        D[v ] ← A[s ][v ]

    WHILE U ≠ V
        D[w ]가 최소인 정점 w ∈ V-U 를 선택
        U ← U ∪ {w }

        FOR w에 인접한 모든 정점 v
                D[v ] ← min(D[v ], D[w ] + A[w ][v ])
```



<그림30_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘17>

D

a	b	c	d	e	f
0	2	3	9	6	10

U

a	b	c	d	e	f

<그림31_SW문제해결 응용_그래프의 최소비용 문제_Dijkstra 알고리즘18>

실습

MST

- 5249. 최소 신장 트리

Dijkstra

- 5250. 최소 비용
- 5251. 최소 이동 거리

추가 연습

- 1795. 인수의 생일 파티

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 Prim 알고리즘의 핵심 아이디어는?

a) 간선을 정렬하고 사이클을 피하면서 선택
b) 시작 정점에서 인접한 간선을 선택해 트리 확장
c) 모든 정점을 방문하는 경로를 우선 선택
d) BFS를 기반으로 방문 순서를 결정

2 Kruskal 알고리즘에서 Union-Find를 사용하는 목적은?

a) 간선의 비용을 빠르게 계산하기 위해
b) 사이클이 생기는지 판단하기 위해
c) 인접 리스트를 만들기 위해
d) DFS 탐색 속도를 높이기 위해

3 w(u, v)가 2일 때, v.key의 최종 값은?

```
u.key = 3
v.key = 5
if w(u, v) < v.key:
    v.key = w(u, v)
```

4 아래 코드에서 출력되는 간선은?

```
edges = [(1, 2, 3), (2, 3, 5), (1, 3, 2)]
edges.sort(key=lambda x: x[2])
print(edges[0])
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 find(2)의 리턴 값은? (초기 상태 기준)

```
parent = [0, 1, 2, 3]
def find(u):
    if parent[u] != u:
        parent[u] = find(parent[u])
    return parent[u]
```

6 다음 중 Prim 알고리즘의 시간 복잡도와 가장 밀접한 자료구조는?

a) 큐
b) 스택
c) 우선순위 큐
d) 해시맵

7 Kruskal 알고리즘의 간선 선택 기준은?

a) 가장 연결된 정점이 많은 간선
b) 인접한 정점만 포함된 간선
c) 가중치가 높은 간선
d) 가중치가 낮은 간선

8 아래 코드에서 선택된 정점 node는?

```
Q = [(0, 'A'), (2, 'B'), (3, 'C')]
Q.sort()
node = Q.pop(0)[1]
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 Prim과 Kruskal의 공통점은?

a) DFS 기반으로 동작한다
b) MST를 구성하기 위한 알고리즘이다
c) 싸이클을 허용한다
d) 간선을 모두 포함한다

10 Kruskal 알고리즘에서 아래 코드의 목적은?

```
A = set()
if find(u) != find(v):
    A.add((u, v))
```

11 Dijkstra 알고리즘은 어떤 문제에 적합한가?

a) MST 문제
b) 최소 간선 수 문제
c) 가중치가 있는 그래프의 최단 경로 문제
d) 음수 간선이 있는 그래프

12 Dijkstra 알고리즘에서 선택된 정점 집합 U의 의미는?

a) 아직 방문하지 않은 정점들
b) 선택 가능한 경로들
c) 현재까지 최단 경로가 확정된 정점들
d) 간선을 저장한 리스트

확인문제

정답 및 해설

1. b) 시작 정점에서 인접한 간선을 선택해 트리 확장 2. b) 싸이클이 생기는지 판단하기 위해 3. 2 4. (1, 3, 2) 5. 2 6. c) 우선순위 큐 7. d) 가중치가 낮은 간선 8. A 9. b) MST를 구성하기 위한 알고리즘이다 10. 싸이클 방지 11. c) 가중치가 있는 그래프의 최단 경로 문제 12. c) 현재까지 최단 경로가 확정된 정점들

1. Prim 알고리즘의 핵심 아이디어는 시작 정점에서 인접한 간선을 선택하여 트리를 확장하는 것입니다.

2. Kruskal 알고리즘에서 Union-Find를 사용하는 목적은 싸이클이 생기는지 판단하기 위함입니다.

3. 2 < 5 이므로 조건이 만족되어 v.key = 2입니다.

4. 가중치 기준 정렬을 활용하여 가장 작은 값은 (1, 3, 2)입니다.

5. 초기 parent[2] = 2 이므로, if parent[2] != 2는 false입니다. 따라서 바로 return 2 가 실행됩니다.

6. 우선순위 큐가 Prim 알고리즘의 시간 복잡도와 가장 밀접합니다.

7. Kruskal 알고리즘은 가중치가 낮은 간선을 기준으로 간선을 선택합니다.

8. Q.sort()에 의해 정렬된 Q는 그대로 유지됩니다.(이미 정렬된 상태) Q.pop(0)에서 (0, 'A'), 즉 [1] 값은 'A'가 됩니다.

9.Prim과 Kruskal 알고리즘의 공통점은 MST를 구성하기 위한 알고리즘이라는 것 입니다.

10. Kruskal 알고리즘에서 두 정점의 루트가 다르면 간선을 추가하고, 루트가 같으면 같은 트리에 이미 속해 있으므로 싸이클이 생깁니다. 즉, 해당 코드는 싸이클을 방지하기 위한 조건입니다.

11. Dijkstra 알고리즘은 가중치가 있는 그래프의 최단 경로 문제에서 적합합니다.

12. Dijkstra 알고리즘에서 선택된 정점 집합 U의 의미는 현재까지 최단 경로가 확정된 정점들입니다.

활동정리

핵심키워드

개념	설명	예시
MST (최소 비용 신장 트리)	무방향 연결 그래프에서 모든 정점을 연결하되, 간선의 가중치 합이 최소인 트리	간선이 6개인 연결 그래프에서 3개의 간선만으로 모든 정점을 연결할 경우
우선순위 큐	최소 또는 최대 값을 빠르게 꺼낼 수 있는 자료구조	Python의 heapq 모듈 등을 활용
싸이클(Cycle)	같은 정점을 다시 방문하는 폐회로 경로	A-B-C-A가 연결되어 있다면 싸이클
가중치(Weight)	간선에 부여된 비용이나 거리 값을 의미	A-B 간선의 가중치가 4이면, 이동 비용이 4라는 뜻
인접 행렬	2차원 배열로 정점 간 연결 상태와 가중치를 표현	A[i][j] = 2는 정점 i와 j가 가중치 2로 연결된 상태

<표1_SW문제해결 응용_그래프의 최소 비용 문제_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **Prim 알고리즘**
  - 시작 정점에서부터 연결된 간선을 하나씩 선택하여 신장 트리를 확장해 나간다.
  - 항상 인접한 간선 중 가장 작은 가중치를 선택한다.

- **Kruskal 알고리즘**
  - 전체 간선을 가중치 기준으로 정렬한 후, 싸이클이 생기지 않도록 하나씩 선택하여 MST를 구성한다.
  - Union-Find 자료구조를 사용한다.

- **Dijkstra 알고리즘**
  - 시작 정점에서 다른 모든 정점까지의 최단 경로를 계산하는 알고리즘으로, 음수 가중치가 없는 경우에만 사용할 수 있다.

활동정리

오늘 공부한 내용 요약 및 정리

- **Union-Find 구조**

 - Kruskal 알고리즘에서 집합을 효율적으로 관리하기 위한 자료구조로, 싸이클 여부를 판단하거나 집합을 합치는 데 쓰인다.

- **최소힙**

 - 우선순위 큐에서 가장 작은 값을 꺼내기 위한 자료구조로, Prim이나 Dijkstra 알고리즘에서 최소 비용 간선을 선택할 때 핵심적으로 사용된다.

- **그래프의 최소비용 문제**

 - 그래프 내에서 특정 조건을 만족하며 가장 적은 비용(가중치)으로 원하는 목적(연결, 경로 등)을 달성하는 문제를 의미한다.

활동정리

'컴퓨터가 바라본 그래프의 최소 비용 문제를 이해해봅시다!'

1. 복잡하게 얽힌 수 많은 정점과 간선을 보고, 전체를 연결하는 가장 [ ] 방법을 찾습니다. 단, 어디에도 [ ] 이 생기면 안됩니다.
2. 때로는 정점에서 출발하고, 어떤 때는 간선을 하나하나 [ ] 하면서 최종적으로 [ ] 이 가장 적은 연결망을 찾습니다.

활동정리

'컴퓨터가 바라본 그래프의 최소 비용 문제를 이해해봅시다!'

1. 복잡하게 얽힌 수 많은 정점과 간선을 보고, 전체를 연결하는 가장 효율적인 방법을 찾습니다. 단, 어디에도 순환(싸이클) 이 생기면 안됩니다.
2. 때로는 정점에서 출발하고, 어떤 때는 간선을 하나하나 선택 하면서 최종적으로 비용이 가장 적은 연결망을 찾습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다